

ФАКУЛЬТЕТ
ПРИКЛАДНОЇ
МАТЕМАТИКИ
ТА ІНФОРМАТИКИ



МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛНУ ІМЕНІ ІВАНА ФРАНКА

CAD-редактор для моделювання гексаедральних сіток з алгоритмом локального згущення

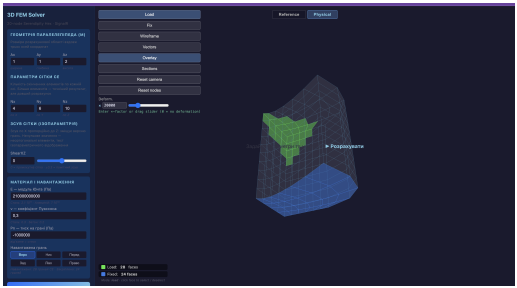
Добрий день, тема моєї курсової роботи — «CAD-редактор для моделювання гексаедральних сіток з алгоритмом локального згущення».

Навіщо це потрібно

- Метод скінченних елементів — основа сучасного інженерного аналізу
- Дозволяє досліджувати напружено-деформований стан, теплообмін, течію
- Без виготовлення реального прототипу

⇒ Але спочатку потрібна сітка об’єкта

⇒ Від якості сітки залежить точність



Комп'ютерне моделювання фізичних процесів — невід'ємна частина сучасного інженерного проектування. Метод скінченних елементів дозволяє досліджувати напружено-деформований стан конструкцій, теплообмін, течію рідини — ще до виготовлення реального прототипу. Це дозволяє виявити небезпечні зони, оцінити запас міцності і уникнути дорогих випробувань.

Але перш ніж виконати будь-який розрахунок — потрібна сітка. Розбиття об'єкта на скінченні елементи. І саме від якості цієї сітки залежить точність результату.

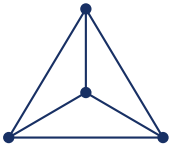
Чому гексаедри

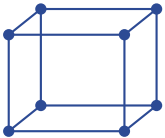
Тетраедри

- Легко генеруються автоматично
- Підходять для будь-якої геометрії
- Нижча точність при тій самій кількості елементів

Гексаедри (шестигранники)

- Складніші у побудові
- Значно точніші
- Краще передають поведінку матеріалу
- Критично важливі для задач



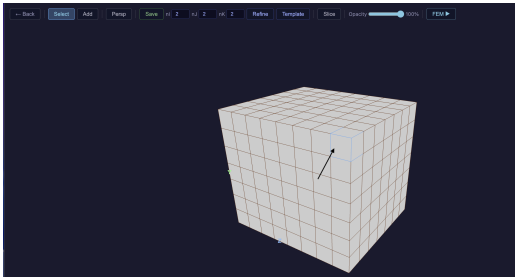


Тривимірна сітка може складатися з різних типів елементів. Найпоширеніший — тетраедри. Для них є багато автоматичних генераторів, вони підходять для будь-якої геометрії. Але тетраедральні сітки дають нижчу точність при тій самій кількості елементів.

Гексаедральні елементи — шестигранники — значно точніші. При однаковій кількості невідомих вони дають меншу похибку. Особливо це критично в задачах механіки деформівного тіла, геомеханіки, аеродинаміки. Тому моя робота зосереджена саме на гексаедральних сітках.

Проблема локального згущення

- Чим дрібніша сітка — тим точніший результат
- Але рівномірне дрібнення всього об'єму: $n \times m \times k$ елементів — нереально
- Потрібно дрібнити лише там де важливо



Ключова складність:

- При поділі елемента на грані з сусідом з'являється новий вузол
- Сусід цього вузла не знає — висячий вузол

Тепер головна проблема. Чим дрібніша сітка — тим точніший результат. Але зони підвищеного інтересу — наприклад, місце звуження деталі або зварний шов — займають малу частину об'єму. Якщо дрібнити сітку рівномірно по всьому об'єкту, кількість елементів зростає в $n \times m \times k$ разів і обчислення стають нереальними.

Тому потрібно локальне згущення — дрібнити лише там де потрібно, а решту залишати грубою.

При такому локальному згущенні гексаедральної сітки виникає ключова складність — висячі вузли. Коли ми ділимо один елемент, на спільній грані з сусідом з'являється новий вузол, якого сусід не знає. Це порушує нерозривність поля переміщень і робить розрахунок некоректним.

Існуючі рішення та їх обмеження

Комерційні (ANSYS, Abaqus, COMSOL, Cubit)

- «Локальне уточнення» — насправді повна перегенерація
- Стара сітка знищується і будується нова
- Від 10 000 до 50 000 $/рік, немає веб-інтерфейсу

Відкриті (Gmsh, Salome, FreeCAD)

- Обмежені переважно тетраедрами
- Для гексаедрів локального уточнення немає

Наукові роботи (Schneiders, Ito та ін.)

- Шаблонне уточнення існує
- Але: каскадне поширення на сусідів
- Погіршення якості в перехідній зоні
- Обмеження: лише бінарний або тернарний поділ
- Жоден не реалізований як доступний веб-інструмент

Ця проблема не нова — над нею працюють з 1990-х років.

Комерційні системи — ANSYS, Abaqus, COMSOL, Cubit — мають потужні алгоритми генерації. Але те що вони називають «локальним уточненням» насправді є повною перегенерацією сітки з іншими параметрами. Стара сітка знищується і будується нова з нуля. Плюс вони коштують від 10 до 50 тисяч доларів на рік і не мають веб-інтерфейсу.

Відкриті системи — Gmsh, Salome, FreeCAD — обмежені переважно тетраедрами. Для гексаедрів локального уточнення в них немає.

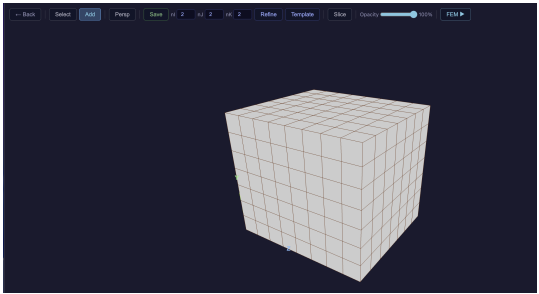
У наукових роботах шаблонне уточнення для гексаедрів існує — Schneiders, Ito, Elsheikh та інші. Але ці підходи мають обмеження: каскадне поширення уточнення на сусідні елементи, погіршення якості в перехідній зоні, обмеження лише бінарним або тернарним поділом. І жоден з них не реалізований як доступний інтерактивний інструмент.

Веб-інтерфейсу для роботи з гексаедральними сітками не існує взагалі.

Мета і завдання

Мета

Розробити та програмно реалізувати інтерактивний веб-редактор гексаедральних сіток із підтримкою локального анізотропного уточнення.



Завдання

1. Розробити алгоритм згущення на основі шаблонів для 3D
2. Вирішити проблему взаємодії між проходами алгоритму
3. Дослідити методи покращення якості сітки

Мета роботи — розробити та програмно реалізувати інтерактивний веб-редактор гексаедральних сіток із підтримкою локального анізотропного уточнення.
Завдання: розробити алгоритм згущення на основі шаблонів для 3D, вирішити проблему взаємодії між проходами алгоритму, дослідити методи покращення якості сітки після уточнення, і реалізувати все це як веб-систему.

Алгоритм згущення

$\perp$

M

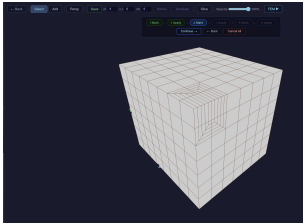
$\rightarrow$

$\rightarrow$

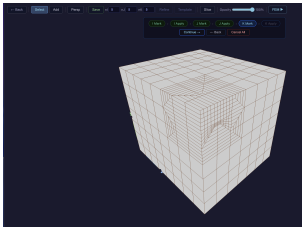
$\rightarrow$

M

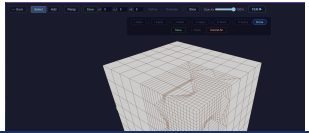
$n \times m \times k$

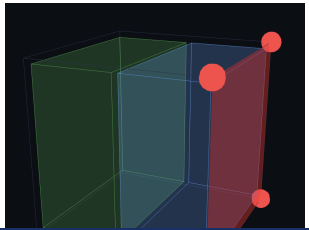


I-прохід



J-прохід





Розглянемо як працює один прохід — наприклад по осі I. Маємо сітку і множину M елементів відмічених для уточнення.

Спочатку ділимо сітку на шари перпендикулярно до осі I. Групуємо шари попарно: 0 і 1, 2 і 3, 4 і 5 і так далі. В кожній парі шарів елементи з'єднуються позиційно — елемент на позиції (j,k) першого шару утворює пару з елементом на тій самій позиції другого шару.

Далі для кожної пари елементів перевіряємо — чи є хоча б один з них у множині M. Якщо ні — пропускаємо. Якщо так — беремо їхню спільну грань і маркуємо на ній 4 вузли.

Так проходимо всі пари. В результаті маємо множину помічених вузлів які належать різним елементам. Кожен елемент може мати 1, 2, 4 або 8 промаркованих вузлів. В залежності від кількості він замінюється відповідним шаблоном: 1 вузол — T1, 2 вузли на ребрі — T2, 4 вузли на грані — T4, 8 — елемент пропускається.

Після застосування шаблонів усі елементи що поділились додаються в множину M якщо їх там ще нема. Далі аналогічно йде прохід по наступній осі.

Усі шаблони узагальнені для довільних параметрів $n \times m \times k$ — це анізотропне уточнення, не лише поділ навпіл.

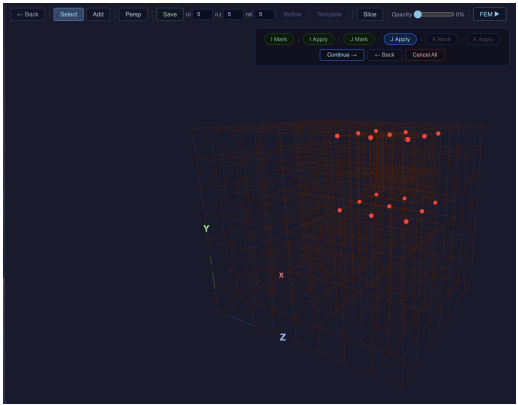
Node Propagation

Проблема

- Наступний прохід зустрічає вже розбитий елемент
- Шаблон напряму застосувати не можна — елемент не листовий
- Без вирішення: Scaled Jacobian падає нижче допустимого

Розв'язок

- Мітки пропагуються вглиб ієрархії до листових нащадків
- Лінійна інтерполяція позицій на кожному рівні





Коли наступний прохід зустрічає елемент що вже був розбитий у попередньому проході — шаблон напряму застосувати не можна, бо елемент вже не листовий. Для цього я розробив механізм Node Propagation. Мітки пропагуються вглиб ієрархії до листових дочірніх елементів через лінійну інтерполяцію позицій. На кожному рівні знаходиться відповідний дочірній і до нього застосовується шаблон.

Це зберігає якість перехідної зони і запобігає каскадному поширенню уточнення на всю сітку. Без цього механізму Scaled Jacobian елементів падає нижче допустимого порогу.

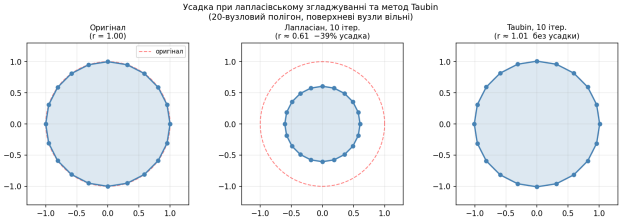
Якість сітки та згладжування

Метрика якості

- Scaled Jacobian $SJ \in [0, 1]$
- $SJ < 0.2$ — проблемний елемент
- $SJ < 0$ — вивернутий елемент
- Теплова карта в реальному часі

Обрано: Taubin

- Лаплас: усуває скошеність, але дає усадку
- Taubin: $\lambda > 0$, потім $\mu < 0$



Після уточнення перехідна зона може містити елементи зі скошеними кутами. Для оцінки якості використовується Scaled Jacobian. Проаналізовано три методи згладжування: лапласівське, Taubin і Taubin Combined. Обрано Taubin — він усуває проблему усадки (стягування сітки до центру при повторному згладжуванні) і покращує форму елементів.

Реалізація системи

Стек

- Backend: .NET 8 — алгоритми, якість, зберігання
- Frontend: Angular 17 + Three.js — 3D у браузері
- Транзакції: будь-яку операцію можна відкотити

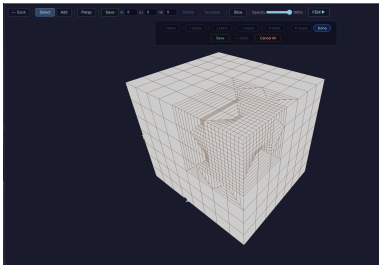


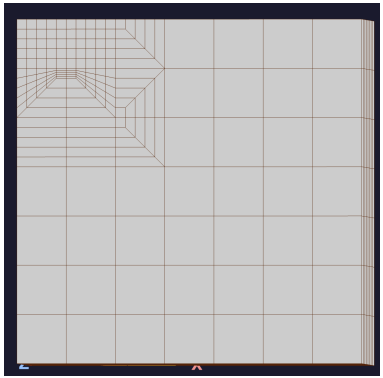
WIZARD

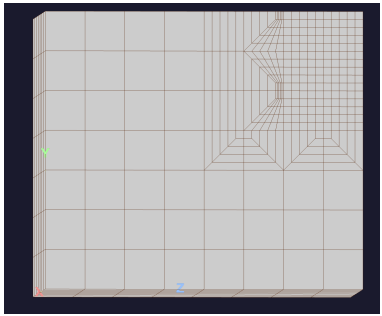
1. Вибір елементів для уточнення
2. Параметри n_1, n_2, n_3
3. Preview результату
4. Підтвердження або відкат

Система CadEditor реалізована як веб-застосунок. Бекенд — .NET 8, фронтенд — Angular 17 з Three.js для 3D-візуалізації. Реалізовано транзакційний механізм — будь-яку операцію уточнення можна відкотити без пошкодження сітки.

Демо — результат 3-прохідного алгоритму







Висновки

Розроблено та реалізовано

1. Алгоритм анізотропного згущення: шаблони T1/T2/T4 для довільних $n \times m \times k$
2. 3-прохідна схема I/J/K з механізмом Node Propagation
3. Дослідження та реалізація методів згладжування
4. Відкрита веб-система з транзакційним механізмом

Перспективи

- Інтеграція MCE-модуля (прототип 80–90% готовий)
- 3D-згладжування методом Taubin
- Імпорт геометричних оболонок з автозаповненням об'єму
- AI-рекомендації параметрів уточнення



Розроблено алгоритм анізотропного згущення гексаедральних сіток з трипрохідною схемою, шаблонами T1/T2/T4 для довільних $n \times m \times k$ і механізмом Node Propagation. Досліджено методи згладжування. Реалізовано як відкриту веб-систему з транзакційним механізмом. Складність алгоритму близька до лінійної.

Інтеграція MCE-модуля — прототип вже готовий на 80–90%. Тривимірне згладжування методом Taubin. Імпорт геометричних оболонок з автозаповненням об'єму. AI-рекомендації параметрів уточнення.

Дякую за увагу!

Готовий відповісти на запитання